Լաբորատոր աշխատանք 11

Օբյեկտի փոխանցումը ֆունկցիային

Աշխատանքի նպատակ.

Տարբերակել օբյեկտի փոխանցումը արժեքով, հղումով և հաստատուն հղումով: Սովորել դասի օբյեկտները փոխանցել արտաքին ֆունկցիաներին և հասկանալ, թե ինչպես է փոխանցման եղանակը ազդում սկզբնական օբյեկտի փոփոխման վրա: Սովորել աշխատել private տվյալների հետ public մեթոդների միջոցով՝ առանց friend ֆունկցիաների օգտագործման:

Խնդրի պահանջ.

Ստեղծել LoanPortfolio դաս՝ private std::vector<double> զանգվածով և p ամսական տոկոսադրույքով: Դասում իրականացնել պարամետրերով կոնստրուկտոր, public ընթերցման մեթոդներ, set(index,value) և print() մեթոդներ: Իրականացնել երեք արտաքին ֆունկցիա՝ analyze(const LoanPortfolio&), changeCopy(LoanPortfolio object, int index, double value) և reversed(const LoanPortfolio&):

analyze() ֆունկցիան պետք է հաշվարկի մեկ ամսվա տոկոսների ընդհանուր գումարը, գտնի ամենամեծ տոկոսագումարով վարկի ինդեքսը և որոշի մայր գումարների միջինից փոքր վարկերի քանակը: changeCopy() ֆունկցիան պետք է փոփոխի միայն իրեն փոխանցված օբյեկտի տեղային պատճենը: reversed() ֆունկցիան պետք է վերադարձնի նոր օբյեկտ՝ վարկերի մայր գումարների շրջված հերթականությամբ և նույն p տոկոսադրույքով:

Տեսական մաս.

Արժեքով փոխանցման դեպքում ֆունկցիային փոխանցվում է օբյեկտի պատճենը: Այդ պատճառով ֆունկցիայի ներսում կատարված փոփոխությունները չեն ազդում սկզբնական օբյեկտի վրա: Այս սկզբունքը կիրառվում է changeCopy() ֆունկցիայում:

Հղումով փոխանցման դեպքում ֆունկցիան աշխատում է սկզբնական օբյեկտի հետ, և փոփոխությունները կարող են ազդել դրա վրա:

const հղումով փոխանցման դեպքում օբյեկտը պատճենելու անհրաժեշտություն չկա, սակայն ֆունկցիան չի կարող փոփոխել փոխանցված օբյեկտը: Այդ եղանակը կիրառվում է analyze() և reversed() ֆունկցիաներում:

Արտաքին սովորական ֆունկցիաները private տվյալներին անմիջականորեն չեն հասնում: Դրանք օգտագործում են դասի public մեթոդները՝ տվյալները կարդալու կամ փոփոխելու համար:

Ընդհանուր պահանջներ.

Տվյալները դասում պետք է պահել private std::vector<double>-ում և p դաշտում: Դինամիկ հիշողության ձեռքով կառավարում այս աշխատանքում չի պահանջվում: Պետք է իրականացնել պարամետրերով կոնստրուկտոր, public ընթերցման մեթոդներ, set(index,value) և print(): Արտաքին ֆունկցիաները պետք է իրականացվեն առանց friend-ի:

Սկզբնական ոչ դատարկ հավաքածուի չափը պետք է լինի 3-ից 30: Բոլոր թվերը պետք է լինեն վերջավոր: Հավասար լավագույն արդյունքների դեպքում պետք է ընտրել առաջին ինդեքսը: Իրական թվերի համեմատության ժամանակ պետք է հաշվի առնել 10^{-9} ճշտությունը:

Սխալ չափով կամ պարամետրերով օբյեկտ չպետք է ստեղծվի, իսկ մուտքի ստուգումը պետք է իրականացվի main()-ում: Փոփոխող մեթոդները նույնպես պետք է ստուգեն թույլատրելիության կանոնները: Սխալ ինդեքսի կամ արժեքի դեպքում set() մեթոդը պետք է վերադարձնի false և պահպանի նախկին տվյալները: Հաշվարկային ֆունկցիաները չպետք է փոփոխեն սկզբնական օբյեկտը:

Վարկերի ամսական տոկոսներ

Ստեղծել LoanPortfolio դաս՝ պահելով վարկերի ընթացիկ մայր գումարների զանգվածը և ամսական p տոկոսադրույքը:

Մեկ ամսվա տոկոսների ընդհանուր գումարը հաշվարկվում է հետևյալ բանաձևով՝ Σ(debt · p / 100): Անհրաժեշտ է գտնել ամենամեծ տոկոսագումարով վարկի ինդեքսը և հաշվել մայր գումարների միջին թվաբանականից փոքր վարկերի քանակը:

Թույլատրելի են միայն դրական մայր գումարներ և 0≤p≤100 տոկոսադրույք: Հաշվարկը մեկ ամսվա պարզ տոկոսի մոդել է՝ առանց մարումների և տույժերի:

Տրված տվյալները.

debt = {200000, 500000, 300000, 800000}

p = 1.5

Ինդեքսները սկսվում են 0-ից:

Հաշվարկներ.

Առաջին վարկի մեկ ամսվա տոկոսը՝ 200000 · 1.5 / 100 = 3000։

Երկրորդ վարկի մեկ ամսվա տոկոսը՝ 500000 · 1.5 / 100 = 7500։

Երրորդ վարկի մեկ ամսվա տոկոսը՝ 300000 · 1.5 / 100 = 4500։

Չորրորդ վարկի մեկ ամսվա տոկոսը՝ 800000 · 1.5 / 100 = 12000։

Տոկոսների ընդհանուր գումարը՝ 3000 + 7500 + 4500 + 12000 = 27000։

Ամենամեծ տոկոսագումարը 12000 է, որը համապատասխանում է 3-րդ ինդեքսին։

Մայր գումարների միջին թվաբանականը՝ (200000 + 500000 + 300000 + 800000) / 4 = 450000։

450000-ից փոքր են 200000 և 300000 արժեքները, հետևաբար դրանց քանակը 2 է։

Իրականացում.

LoanPortfolio դասում private բաժնում սահմանվում է std::vector<double> data զանգվածը և double p տոկոսադրույքը։ Պարամետրերով կոնստրուկտորը ստանում է վարկերի մայր գումարների զանգվածը և տոկոսադրույքը։

Public մեթոդների միջոցով հնարավոր է ստանալ զանգվածի չափը, առանձին արժեքը և p տոկոսադրույքը։ set(index,value) մեթոդը փոփոխում է նշված արժեքը միայն այն դեպքում, երբ ինդեքսը և նոր արժեքը վավեր են։ print() մեթոդը արտածում է օբյեկտի տվյալները։

analyze(const LoanPortfolio&) ֆունկցիան օբյեկտը ստանում է հաստատուն հղումով։ Այն հաշվարկում է բոլոր վարկերի մեկ ամսվա տոկոսները, գտնում է ամենամեծ տոկոսագումարի առաջին ինդեքսը, հաշվում է միջին մայր գումարը և դրանից փոքր արժեքների քանակը։

changeCopy(LoanPortfolio object, int index, double value) ֆունկցիան օբյեկտը ստանում է արժեքով։ Ֆունկցիայի ներսում փոփոխվում է միայն տեղային պատճենը, իսկ սկզբնական օբյեկտը մնում է անփոփոխ։

reversed(const LoanPortfolio&) ֆունկցիան օբյեկտը ստանում է const հղումով, ստեղծում է նոր օբյեկտ և դրա տվյալները դասավորում հակառակ հերթականությամբ։ Սկզբնական օբյեկտը չի փոփոխվում։

Փորձնական ստուգում.

changeCopy() ֆունկցիան կանչելիս կարելի է փոխանցել index=1 և value=600000։ Այդ դեպքում տեղային պատճենը կդառնա {200000, 600000, 300000, 800000}։ Նոր տոկոսների

ընդհանուր գումարը կլինի 28500, սակայն սկզբնական օբյեկտը կպահպանի {200000, 500000, 300000, 800000} արժեքները։

reversed() ֆունկցիայի արդյունքը կլինի {800000, 300000, 500000, 200000}, իսկ p=1.5 արժեքը կմնա նույնը։ Սկզբնական օբյեկտը կրկին չի փոփոխվի։

Առաջադրանք.

Կազմել C++ ծրագիր, որը կկիրառի LoanPortfolio դասը՝ վարկերի ընթացիկ մայր գումարների և ամսական տոկոսադրույքի պահպանման համար։ Ծրագիրը պետք է օգտագործի private std::vector<double> զանգված և p դաշտ, իրականացնի պարամետրերով կոնստրուկտոր, public ընթերցման մեթոդներ, set() և print() մեթոդներ։

Անհրաժեշտ է իրականացնել analyze(const LoanPortfolio&), changeCopy(LoanPortfolio object, int index, double value) և reversed(const LoanPortfolio&) արտաքին ֆունկցիաները՝ առանց friend-ի։ Ծրագիրը պետք է ստուգի սխալ մուտքերը, չփոխի սկզբնական տվյալները հաշվարկների ժամանակ և ցուցադրի արժեքով ու const հղումով փոխանցման տարբերությունը։

Փորձարկման համար օգտագործել debt={200000,500000,300000,800000}, p=1.5 տվյալները։ Ծրագիրը պետք է արտածի մեկ ամսվա տոկոսների ընդհանուր գումարը, ամենամեծ տոկոսագումարով վարկի ինդեքսը, միջինից փոքր մայր գումարների քանակը, փոփոխված պատճենի տվյալները և շրջված նոր օբյեկտը՝ յուրաքանչյուր գործողությունից հետո ցուցադրելով նաև սկզբնական օբյեկտը։

```
#include <iostream>
#include <vector>
#include <cmath>
using namespace std;
class LoanPortfolio {
private:
    vector<double> data;
    double p;
public:
    // Պարամետրերով կոնստրուկտոր
    LoanPortfolio(const vector<double>& values, double percent) {
        if (values.size() < 3 || values.size() > 30) {
            p = 0;
```

```
        return;
    }
    if (!isfinite(percent) || percent < 0 || percent > 100) {
        p = 0;
        return;
    }
    for (int i = 0; i < (int)values.size(); i++) {
        if (!isfinite(values[i]) || values[i] <= 0) {
            data.clear();
            p = 0;
            return;
        }
    }
    data = values;
    p = percent;
}
// Տարրերի քանակը
int size() const {
    return (int)data.size();
}
// Տարրի ստացում
double get(int index) const {
    return data[index];
}
// Տոկոսադրույքի ստացում
double getP() const {
    return p;
}
// Տարրի փոփոխում
bool set(int index, double value) {
    if (index < 0 || index >= size()) {
```

```
        return false;
    }
    if (!isfinite(value) || value <= 0) {
        return false;
    }
    data[index] = value;
    return true;
}
// Տվյալների արտածում
void print() const {
    cout << "Loans: {";
    for (int i = 0; i < size(); i++) {
        cout << data[i];
        if (i < size() - 1) {
            cout << ", ";
        }
    }
    cout << "}" << endl;
    cout << "p = " << p << "%" << endl;
}
// Զանգվածի շրջում
void reverse() {
    int left = 0;
    int right = size() - 1;
    while (left < right) {
        double temp = data[left];
        data[left] = data[right];
        data[right] = temp;
        left++;
        right--;
    }
```

```
    }
};
// Const հղումով փոխանցում
void analyze(const LoanPortfolio& object) {
    double totalInterest = 0;
    double sum = 0;
    int maxIndex = 0;
    double maxInterest = object.get(0) * object.getP() / 100.0;
    // Տոկոսների գումար և առավելագույն տոկոս
    for (int i = 0; i < object.size(); i++) {
        double interest = object.get(i) * object.getP() / 100.0;
        totalInterest += interest;
        sum += object.get(i);
        if (interest > maxInterest + 1e-9) {
            maxInterest = interest;
            maxIndex = i;
        }
    }
    // Մայր գումարների միջինը
    double average = sum / object.size();
    // Միջինից փոքր վարկերի քանակը
    int belowAverage = 0;
    for (int i = 0; i < object.size(); i++) {
        if (object.get(i) < average - 1e-9) {
            belowAverage++;
        }
    }
    cout << "Մեկ ամսվա տոկոսների գումար = "
         << totalInterest << endl;
    cout << "Ամենամեծ տոկոսագումարով վարկի ինդեքս = "
         << maxIndex << endl;
```

```
    cout << "Մայր գումարների միջինից փոքր վարկերի քանակ = "
         << belowAverage << endl;
}
// Արժեքով փոխանցում
void changeCopy(LoanPortfolio object, int index, double value) {
    if (object.set(index, value)) {
        cout << "Փոփոխված պատճենի տվյալները:" << endl;
        object.print();
        analyze(object);
    }
    else {
        cout << "Սխալ ինդեքս կամ արժեք:" << endl;
    }
}
// Const հղումով փոխանցում
LoanPortfolio reversed(const LoanPortfolio& object) {
    LoanPortfolio result(object);
    result.reverse();
    return result;
}
int main() {
    vector<double> debt = {
        200000,
        500000,
        300000,
        800000
    };
    double p = 1.5;
    // Մուտքային տվյալների ստուգում
    bool valid = true;
```

```
if (debt.size() < 3 || debt.size() > 30) {
    valid = false;
}
if (!isfinite(p) || p < 0 || p > 100) {
    valid = false;
}
for (int i = 0; i < (int)debt.size(); i++) {
    if (!isfinite(debt[i]) || debt[i] <= 0) {
        valid = false;
    }
}
if (!valid) {
    cout << "Սխալ մուտքային տվյալներ:" << endl;
    return 0;
}
LoanPortfolio portfolio(debt, p);
// Բնօրինակ օբյեկտ
cout << "Բնօրինակ օբյեկտը:" << endl;
portfolio.print();
analyze(portfolio);
// changeCopy()
cout << endl;
cout << "changeCopy() ֆունկցիայի կանչից հետո:" << endl;
changeCopy(portfolio, 1, 600000);
// Ստուգում ենք, որ բնօրինակը չի փոխվել
cout << endl;
cout << "Բնօրինակը changeCopy()-ից հետո:" << endl;
portfolio.print();
analyze(portfolio);
```

```
    // reversed()
    cout << endl;
    cout << "reversed() ֆունկցիայի արդյունքը:" << endl;
    LoanPortfolio reversedPortfolio = reversed(portfolio);
    reversedPortfolio.print();
    analyze(reversedPortfolio);
    // Բնօրինակի ստուգում
    cout << endl;
    cout << "Բնօրինակը reversed()-ից հետո:" << endl;
    portfolio.print();
    analyze(portfolio);
    return 0;
}
```